Unsupervised Learning and Dimensionality Reduction

Contents

1

Unsupervised Learning

Introduction

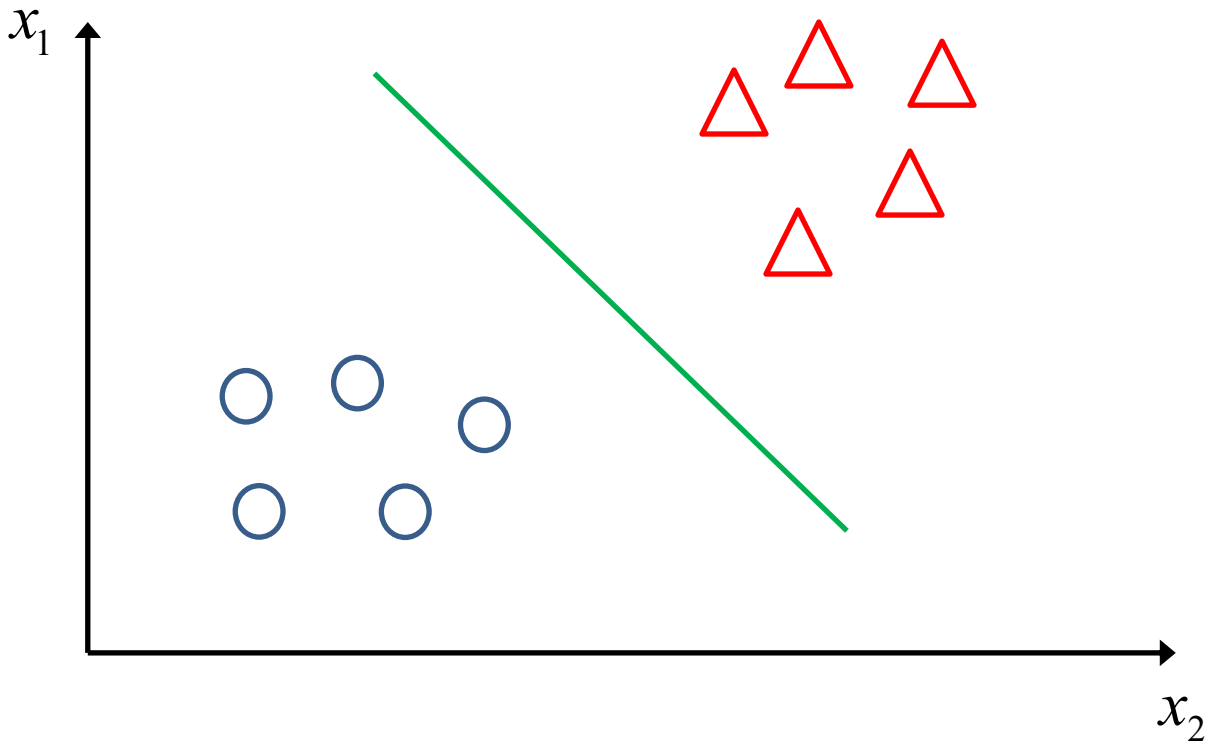
- Unsupervised (self-organized) learning finds patterns in data without using a teacher or labeled examples.

- The algorithm updates weights using local rules, meaning each neuron only adjusts based on its nearby connections.

1 Unsupervised Learning (Clustering Problem)

❖ **Introduction into unsupervised learning**

- Supervised learning example: Classification Problem

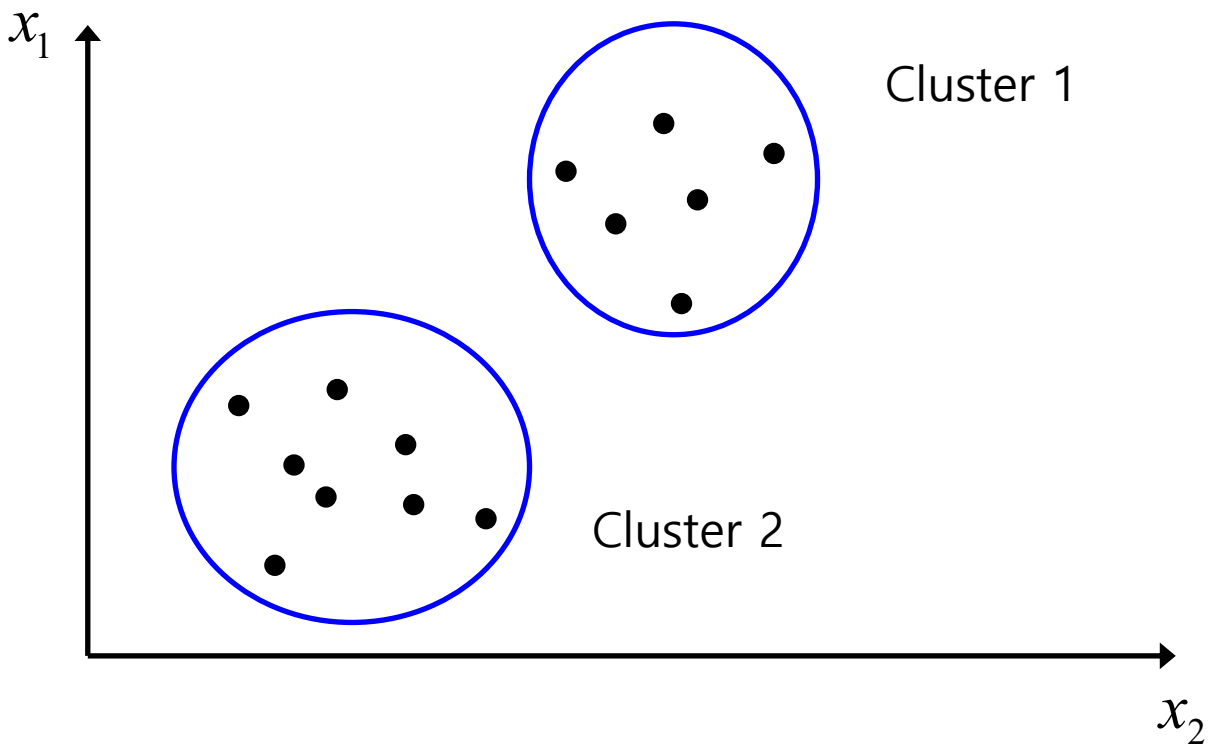


Given a training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}), \ldots, (x^{(m)}, y^{(m)})\}$

1 Unsupervised Learning (Clustering Problem)

❖ **Introduction into unsupervised learning**

- Unsupervised learning example: Clustering Problem

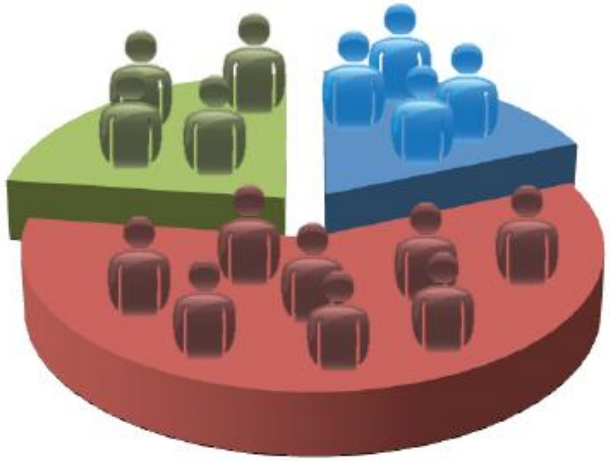


Given a training set: $\{x^{(1)}, x^{(2)}, x^{(3)}, ..., x^{(m)}\}$

1 Unsupervised Learning (Clustering Problem)

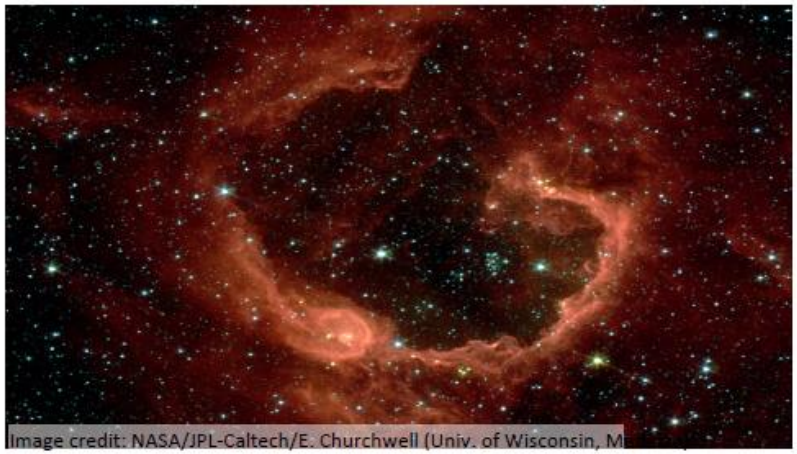
❖ Introduction into unsupervised learning

- Applications of clustering



Market Segmentation

Organize computing clusters



Astronomical data analysis

2

K-means Clustering Algorithm

2 K-means clustering algorithm

❖ K-means algorithm

- Inputs:
  — K (number of clusters)

  — Training set $\{x^{(1)}, x^{(2)}, x^{(3)}, \ldots, x^{(m)}\}, x^{(i)} \in \mathbb{R}^n$

- Algorithm flow:

  Randomly initialize K cluster centroids $\mu_1, \mu_2, \ldots, \mu_K \in \mathbb{R}^n$

  Repeat {

  for *i* = 1 to *m*

  $c^{(i)}$:= index (from 1 to *K*) of cluster centroid closest to $x^{(i)}$

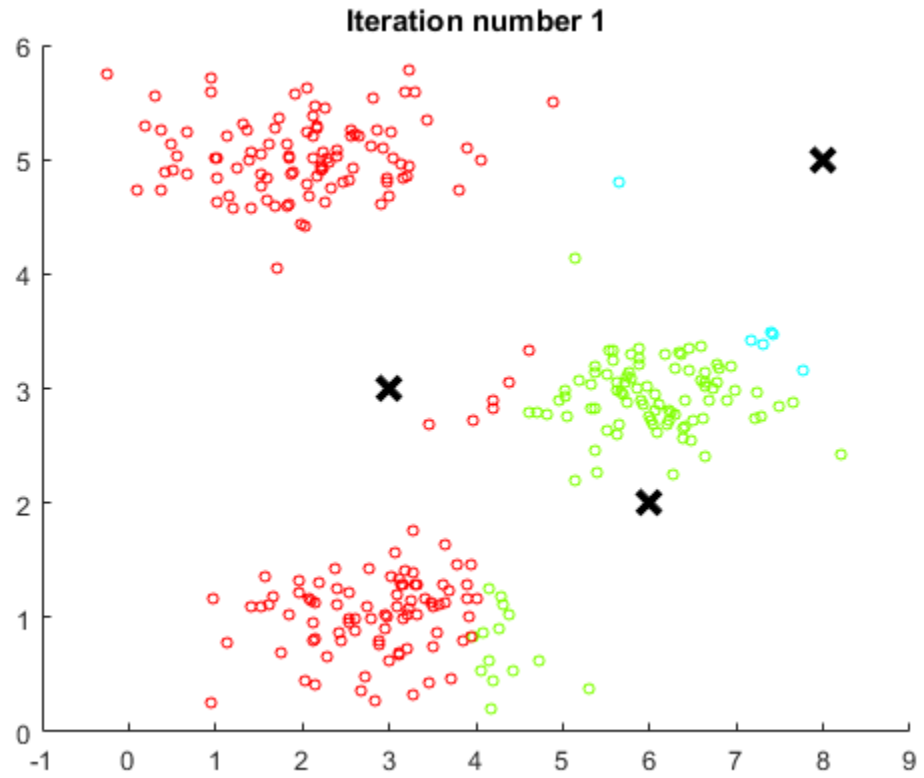
  for *k* = 1 to *K*

  μ_k := average (mean) of points assigned to cluster *k*

  }

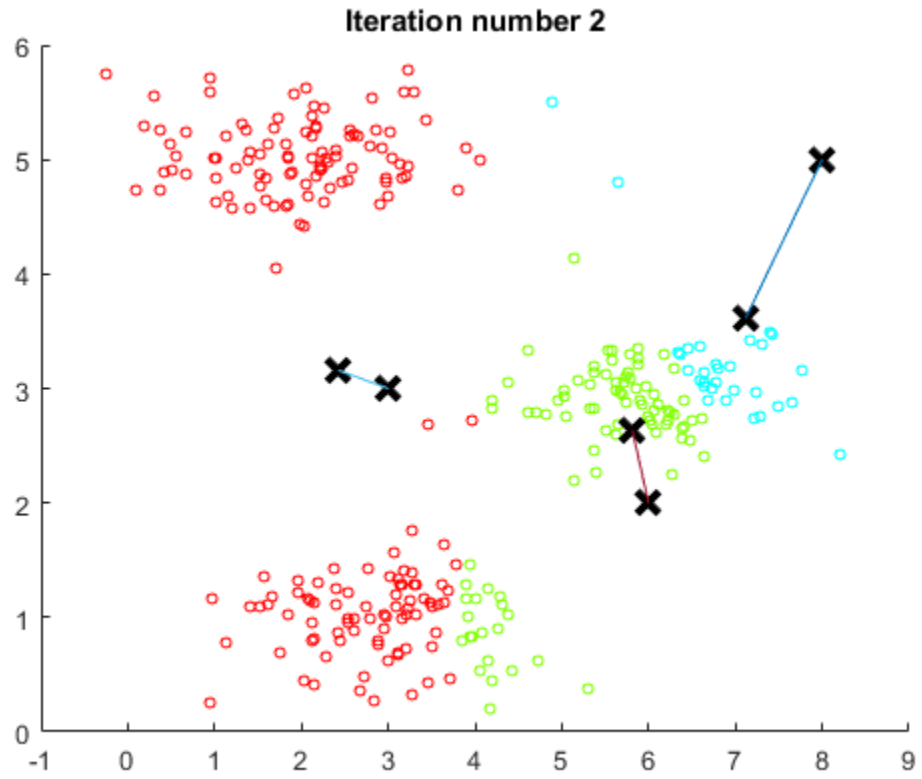
❖ K-means algorithm

- Execution of algorithm



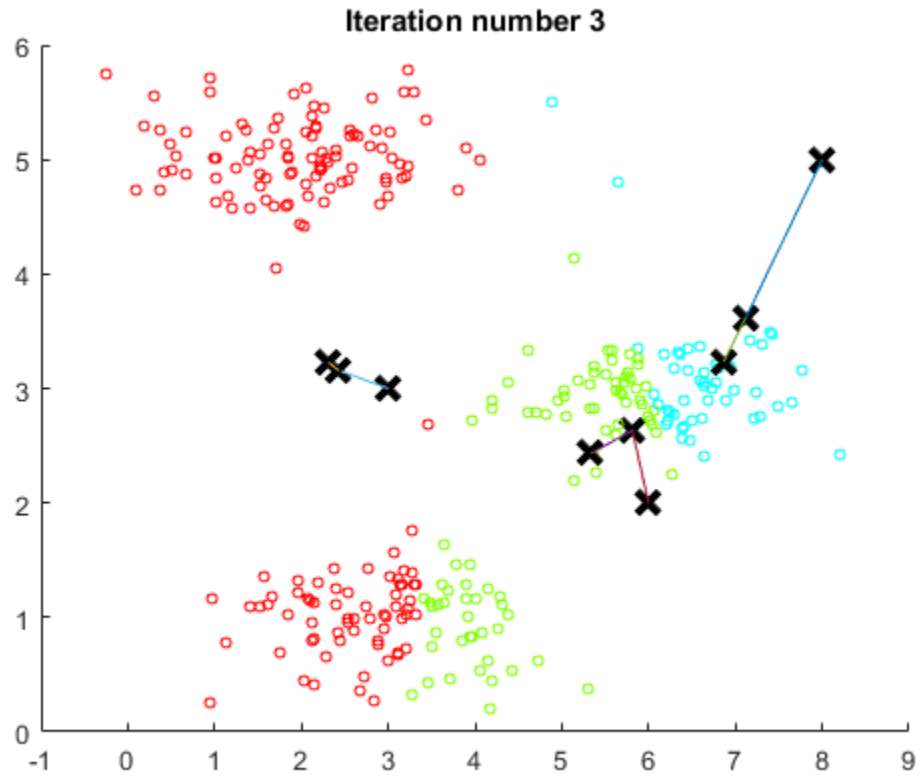
❖ K-means algorithm

- Execution of algorithm



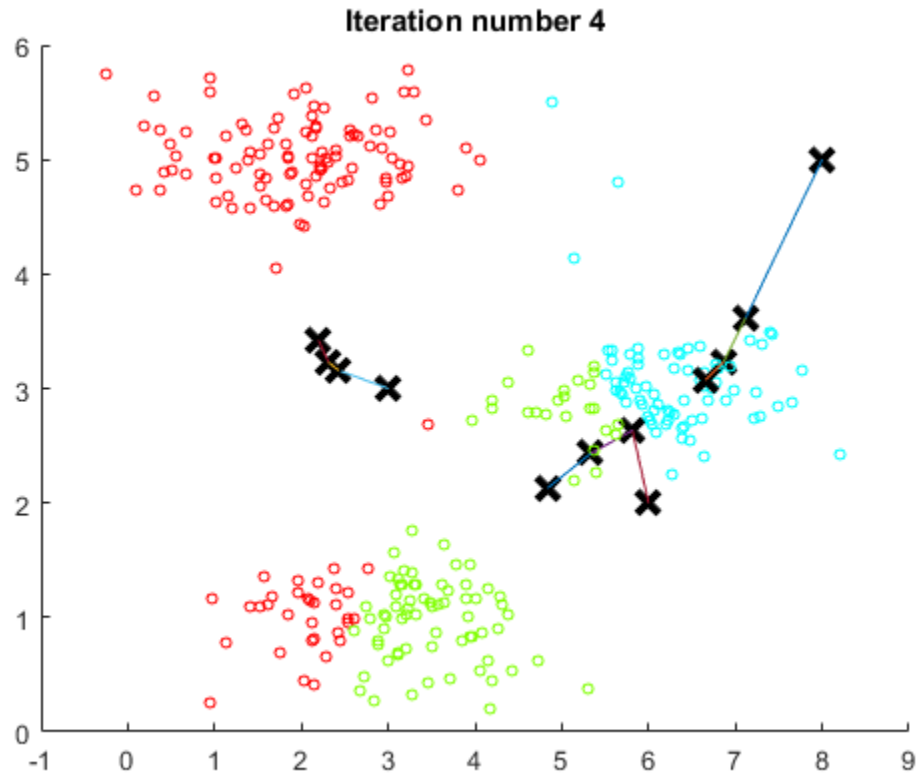
❖ K-means algorithm

- Execution of algorithm



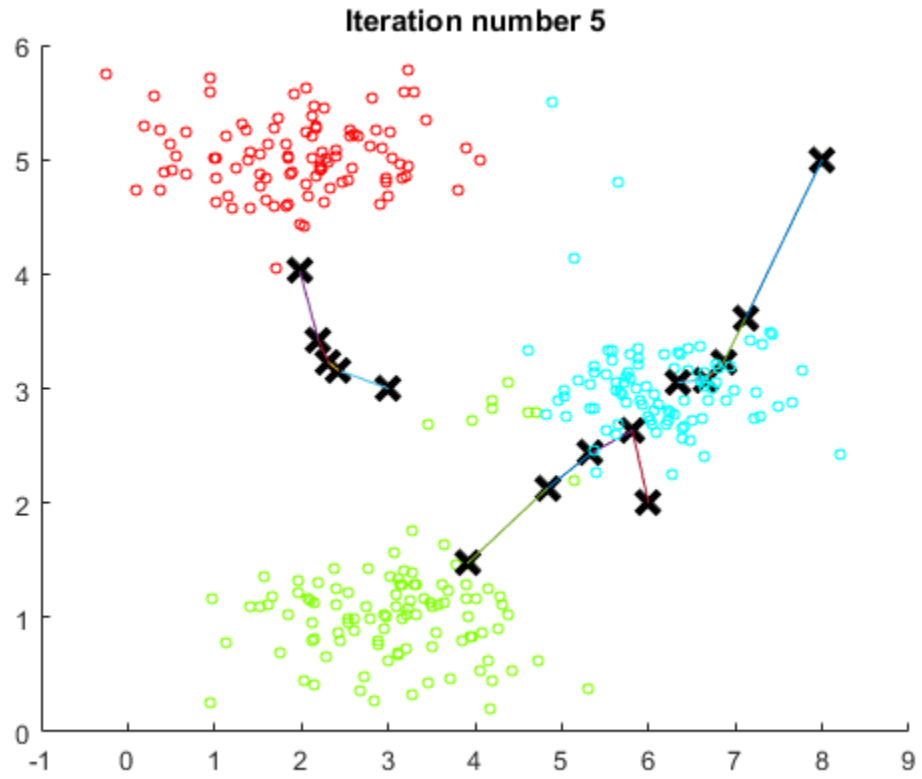
❖ K-means algorithm

- Execution of algorithm



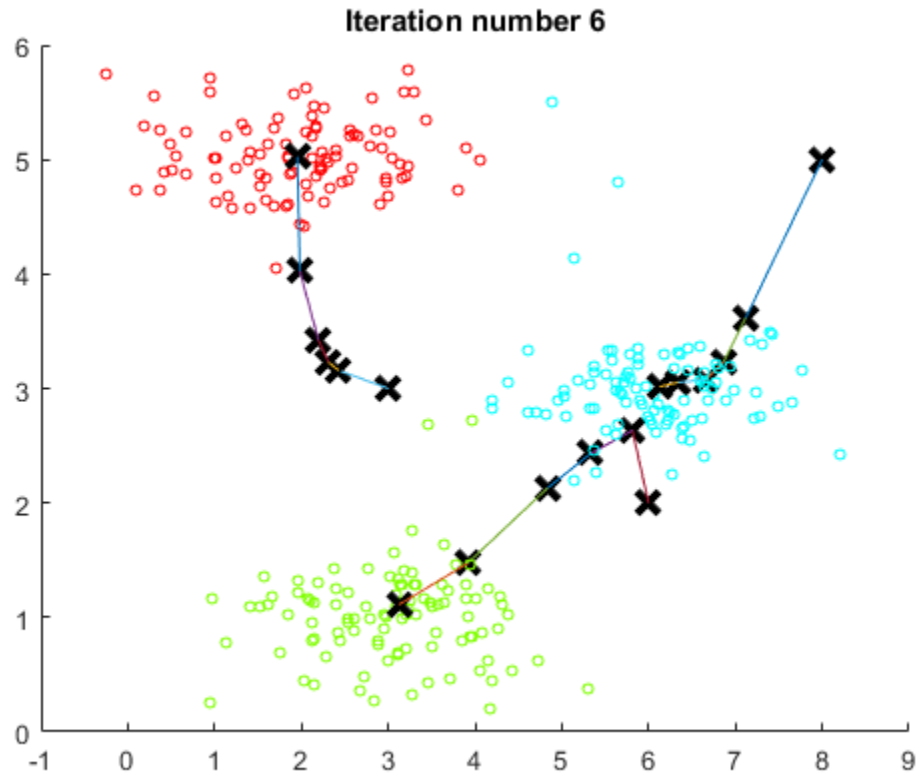
❖ K-means algorithm

- Execution of algorithm



❖ K-means algorithm

- Execution of algorithm



❖ K-means algorithm objective function

$c^{(i)}$ =index of cluster (1,2,...,K) to which example $x^{(i)}$ is currently assigned

μ_k =cluster centroid k $(\mu_k \in \mathbb{R}^n)$

$\mu_{c^{(i)}}$ =cluster centroid to which example $x^{(i)}$ has been assigned

$$J(c^{(1)},\ldots,\mathrm{c}^{(m)},\mu_1,\ldots,\mu_K)=\frac{1}{m}\sum_{i=1}^{m}\left\|x^{(i)}-\mu_{c^{(i)}}\right\|^2$$

$$\min_{\substack{c^{(1)},\ldots,c^{(m)}\\ \mu_1,\ldots,\mu_K}} J(c^{(1)},\ldots,\mathrm{c}^{(m)},\mu_1,\ldots,\mu_K)$$

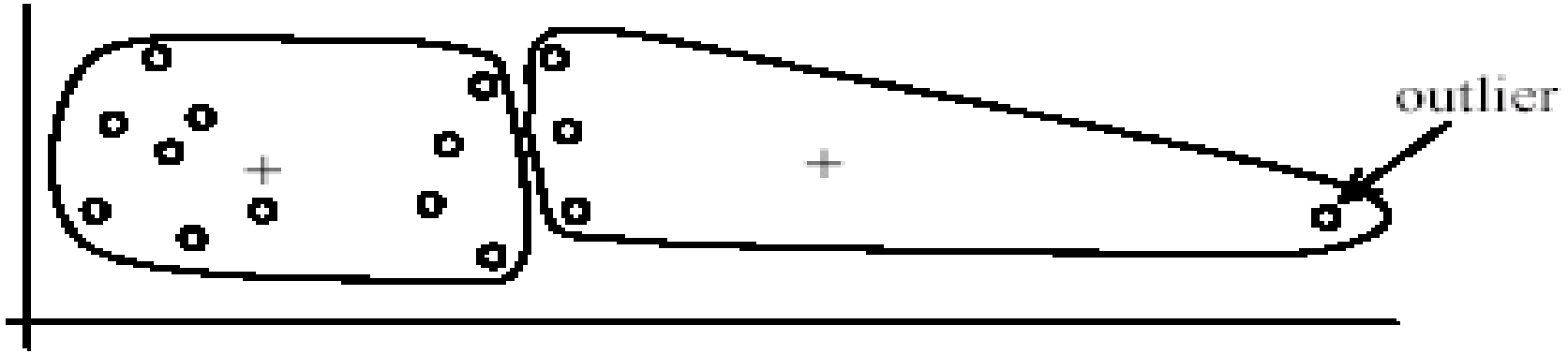
Strengths of k-means

- Strengths:
  - Simple: easy to understand and to implement
  - Efficient: Time complexity: $O(tkn)$,
    where n is the number of data points,
    k is the number of clusters, and
    t is the number of iterations.
  - Since both k and t are small. k-means is considered a linear algorithm.
- K-means is the most popular clustering algorithm.
- Note that: it terminates at a local optimum if Sum of Squared Errors (SSE) is used. The global optimum is hard to find due to complexity.

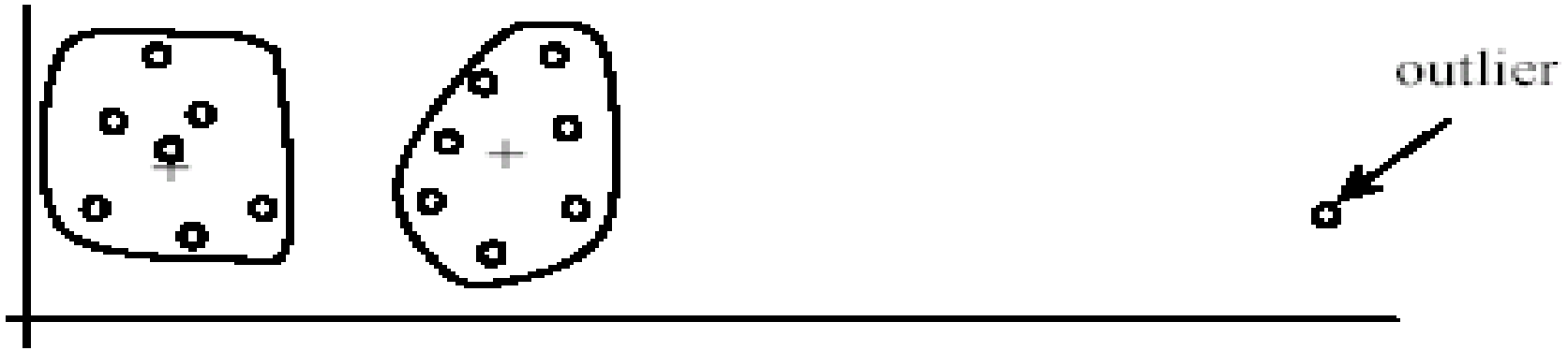
Weaknesses of k-means

- The algorithm is only applicable if the mean is defined.
  - For categorical data, *k*-mode - the centroid is represented by most frequent values.
- The user needs to specify *k*.
- The algorithm is sensitive to **outliers**
  - Outliers are data points that are very far away from other data points.
  - Outliers could be errors in the data recording or some special data points with very different values.

Weaknesses of k-means: Problems with outliers



(A): Undesirable clusters



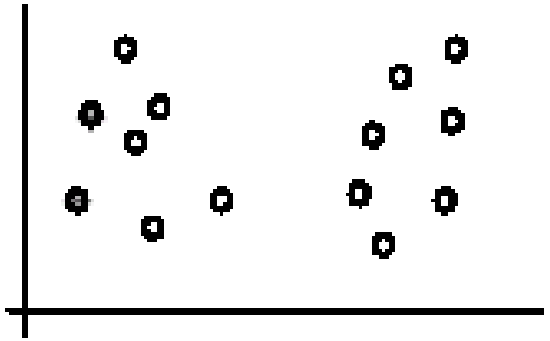
(B): Ideal clusters

Weaknesses of k-means: To deal with outliers

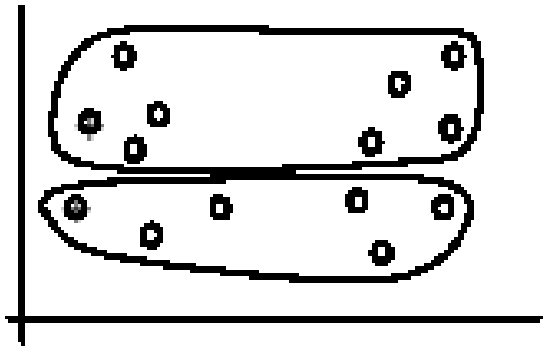
- One method is to remove some data points in the clustering process that are much further away from the centroids than other data points.
  - To be safe, we may want to monitor these possible outliers over a few iterations and then decide to remove them.
- Another method is to perform random sampling. Since in sampling we only choose a small subset of the data points, the chance of selecting an outlier is very small.
  - Assign the rest of the data points to the clusters by distance or similarity comparison, or classification

Weaknesses of k-means (cont ...)

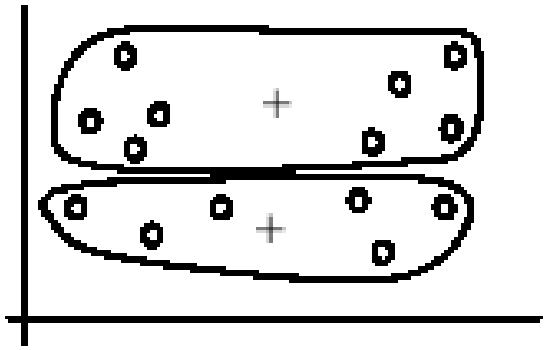
- The algorithm is sensitive to initial seeds.



(A). Random selection of seeds (centroids)



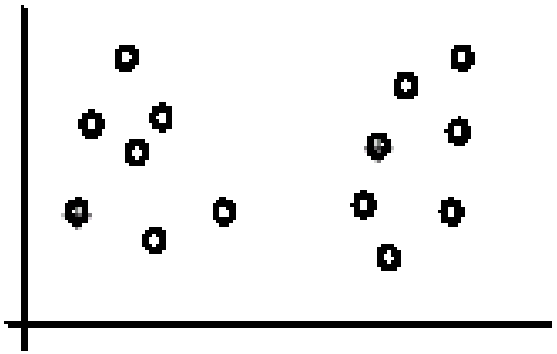
(B). Iteration 1



(C). Iteration 2

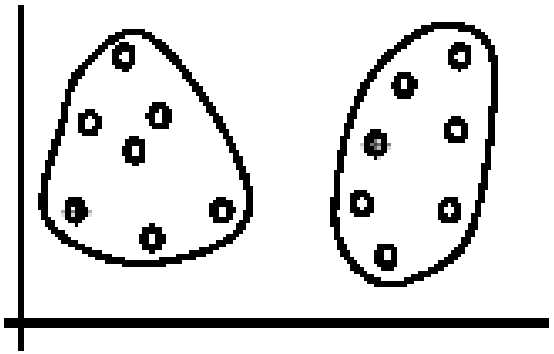
Weaknesses of k-means (cont …)

- If we use different seeds: good results

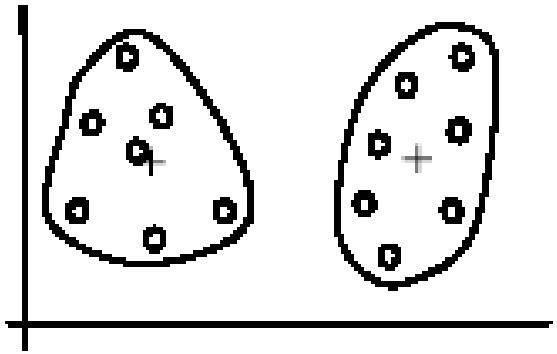


(A). Random selection of k seeds (centroids)

There are some methods to help choose good seeds



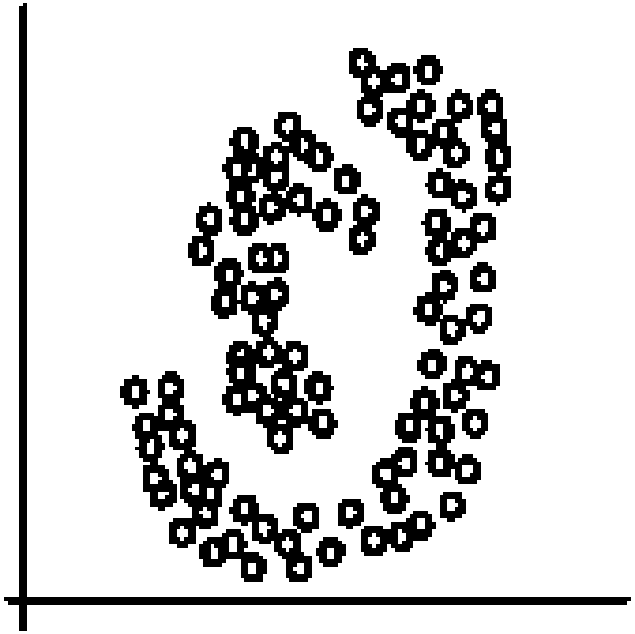
(B). Iteration 1



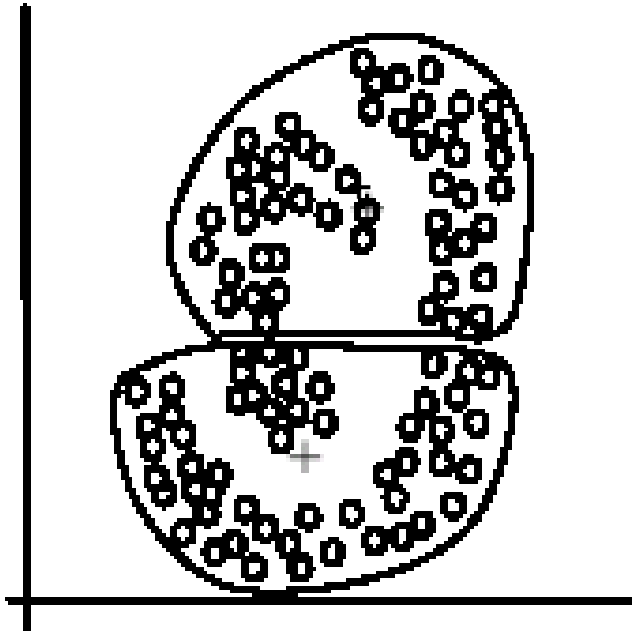
(C). Iteration 2

Weaknesses of k-means (cont.)

- The *k*-means algorithm is not suitable for discovering clusters that are hyper-ellipsoids (or hyper-spheres).



(A): Two natural clusters

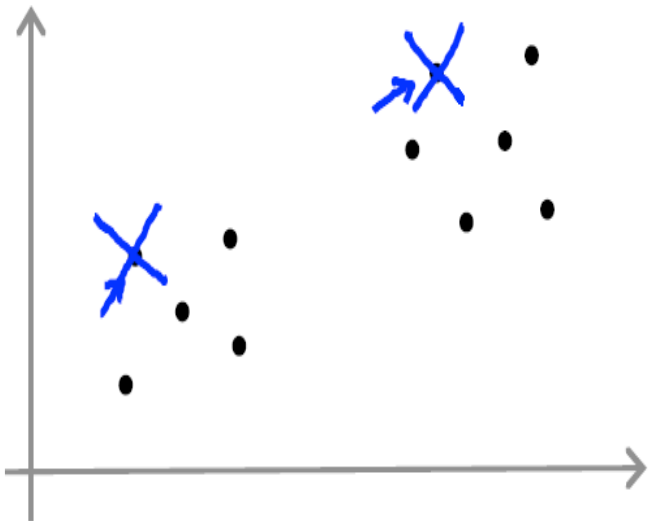


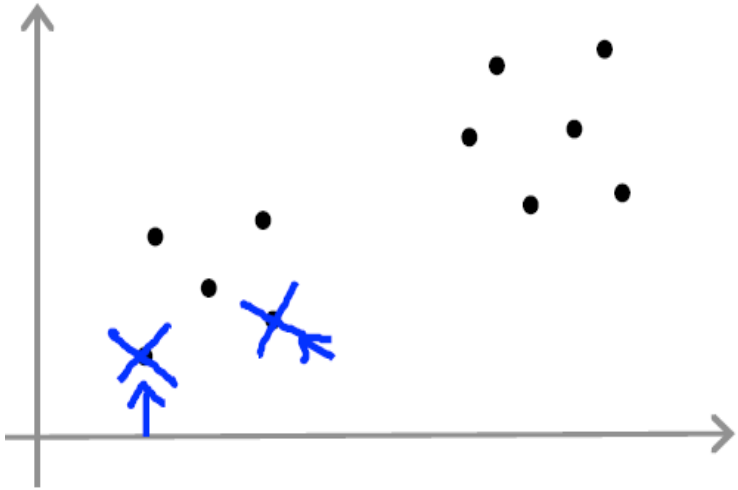
(B): *k*-means clusters

3 Centroid initialization and choosing the number of K

❖ **Random initialization of centroids for K-means algorithm**

- Number of clusters K<m
- Randomly pick K training examples
- Set $\mu_1, \ldots, \mu_K$ equal to these K examples



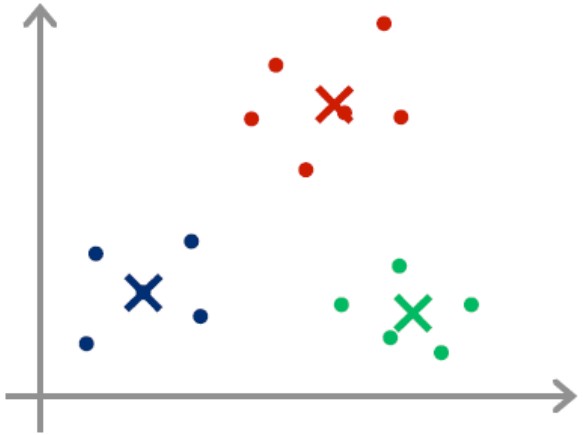


3 Centroid initialization and choosing the number of K

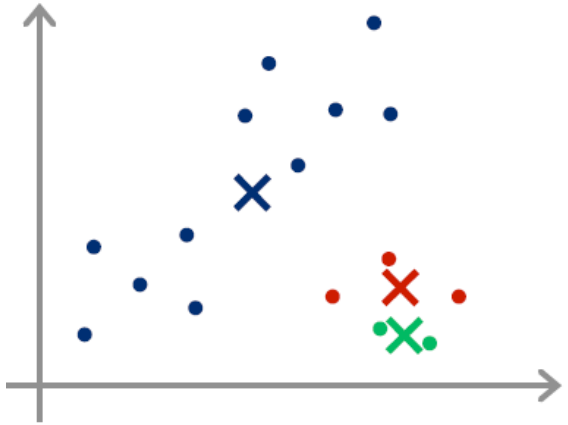
❖ **Random initialization of centroids for K-means algorithm**

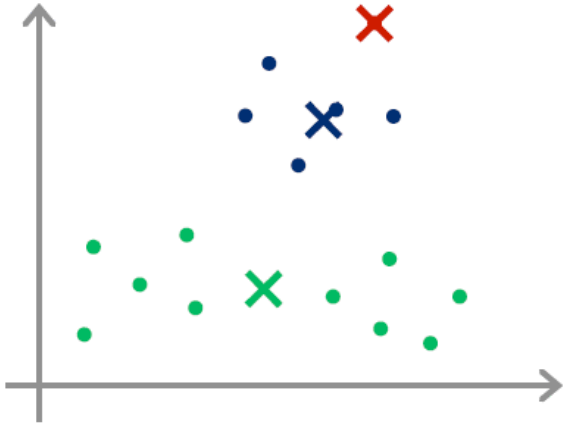
- Problem

Ideal case of random initialization



Bad examples of random initialization





3 Centroid initialization and choosing the number of K

❖ **Random initialization of centroids for K-means algorithm**

- Solution

For i =1 to 100 {
Randomly initialize K-means.
Run K-means. Get $c^{(1)},\ldots,c^{(m)},\mu_1,\ldots,\mu_K$
Compute cost function:

$$J(c^{(1)},\ldots,c^{(m)},\mu_1,\ldots,\mu_K)$$

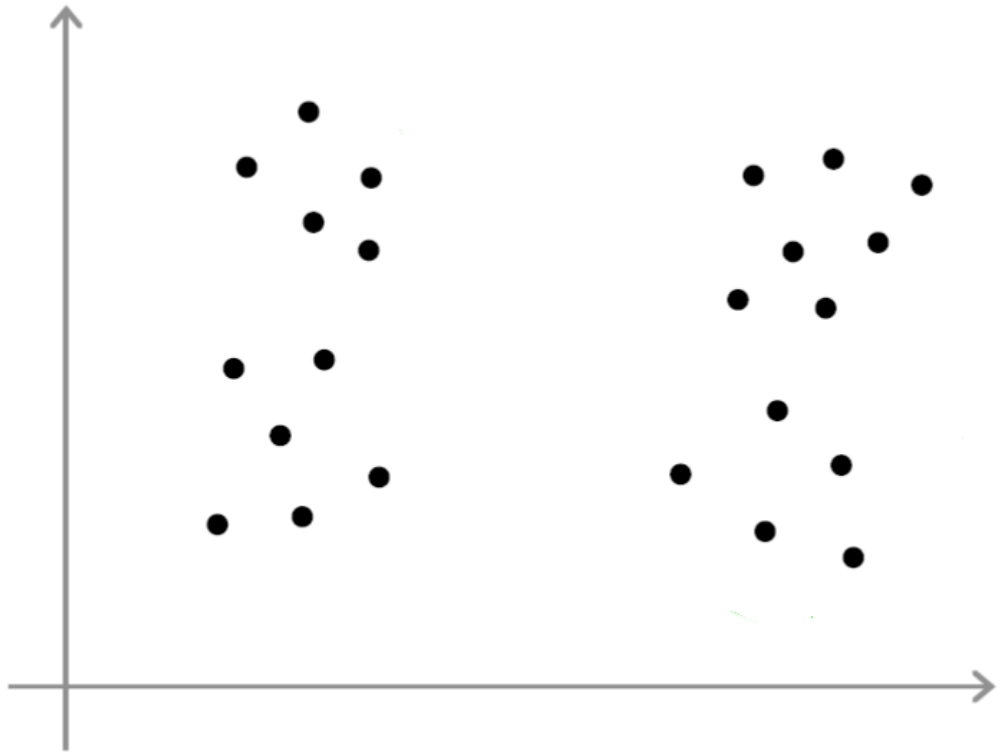
}

Pick clustering with the lowest cost $J(c^{(1)},\ldots,c^{(m)},\mu_1,\ldots,\mu_K)$

3 Centroid initialization and choosing the number of K

❖ Choosing the number of clusters

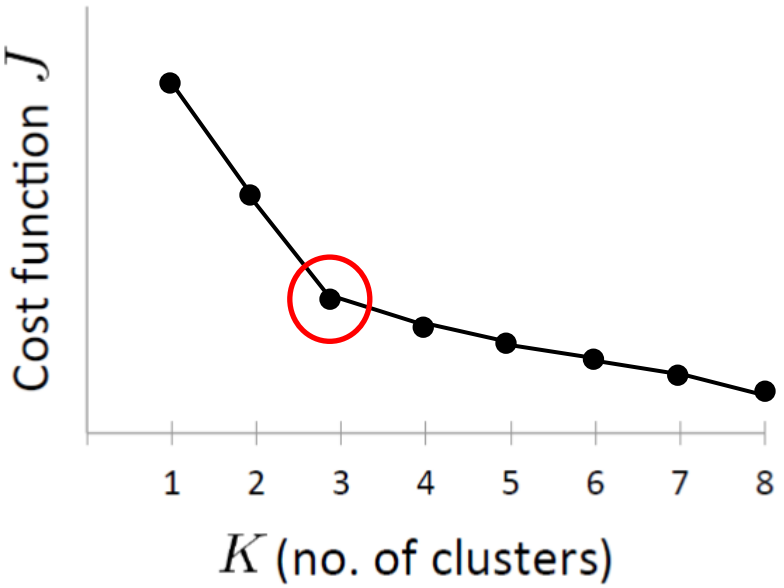
- What is the right number of clusters? How to choose it?



3 Centroid initialization and choosing the number of K

❖ **Choosing the number of clusters K**

- Elbow method



3 Example of K Means Clustering

- **Dataset:**
- We have the following 10 data points:

  (2,10),(2,5),(8,4),(5,8),(7,5),(6,4),(1,2),(4,9),(7,3),(6,8)

Step 1: Initialize k and centroids
Set k=2 (we want 2 clusters).
Randomly initialize 2 centroids: C1=(2,10), C2=(5,8)

Step 2: Assign data points to the nearest centroid

For each data point, calculate the Euclidean distance to each centroid. Assign the point to the cluster with the nearest centroid.

Euclidean Distance Formula:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3 Example of K Means Clustering

- **Dataset:**
- We have the following 10 data points:

  (2,10),(2,5),(8,4),(5,8),(7,5),(6,4),(1,2),(4,9),(7,3),(6,8)

For each data point:

1. (2, 10):
   - Distance to $C_1 = (2, 10)$: $\sqrt{(2-2)^2 + (10-10)^2} = 0$
   - Distance to $C_2 = (5, 8)$: $\sqrt{(5-2)^2 + (8-10)^2} = 3.61$
   - Assign to C_1.
2. (2, 5):
   - Distance to $C_1 = (2, 10)$: 5
   - Distance to $C_2 = (5, 8)$: 4.24
   - Assign to C_2.

(Similar calculations are performed for all points.)

3 Example of K Means Clustering

- **Dataset:**
- We have the following 10 data points:

  (2,10),(2,5),(8,4),(5,8),(7,5),(6,4),(1,2),(4,9),(7,3),(6,8)

Step 3: Update centroids
For each cluster, calculate the new centroid by taking the mean of all points assigne d to that cluster.
For C1: Mean of points assigned to C1.
For C2: Mean of points assigned to C2.

Step 4: Repeat until convergence
Repeat Steps 2 and 3 until the centroids no longer change significantly.

3 Example of K Means Clustering: Python Code

https://colab.research.google.com/drive/1R5TEaFIMV8dRVLH33qcUi8DvuVCyGFUZ?usp=sharing

3

Dimensionality Reduction

Motivation

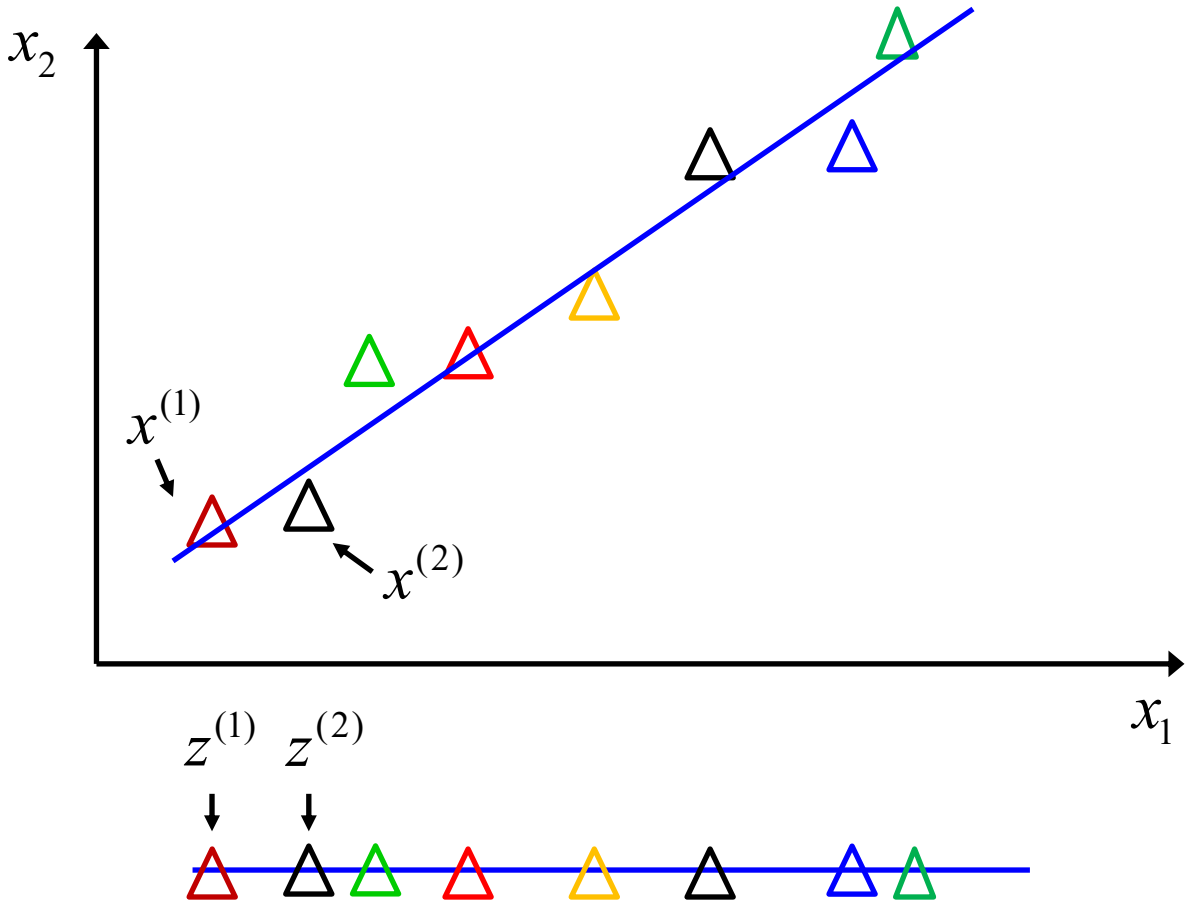
Dimensionality reduction is the process of reducing the number of in put variables, or features, in a dataset while retaining as much meanin gful information as possible. It's a crucial technique in machine learnin g and data analysis.

Why do we need it?

- **The "Curse of Dimensionality":** As the number of features increa ses, the amount of data needed to support a robust model grows ex ponentially. High-dimensional data is sparse, making it difficult for al gorithms to find patterns.
- **Efficiency:** Fewer features mean less computation time and memo ry, making models faster to train.
- **Simpler Models:** Models with fewer features are easier to interpret and less prone to overfitting.

❖ **Motivation. Data Compression and Visualization.**

- Reduce data from 2D to 1D



Mapping

$x^{(1)} \in \mathbb{R}^2 \rightarrow z^{(1)} \in \mathbb{R}$

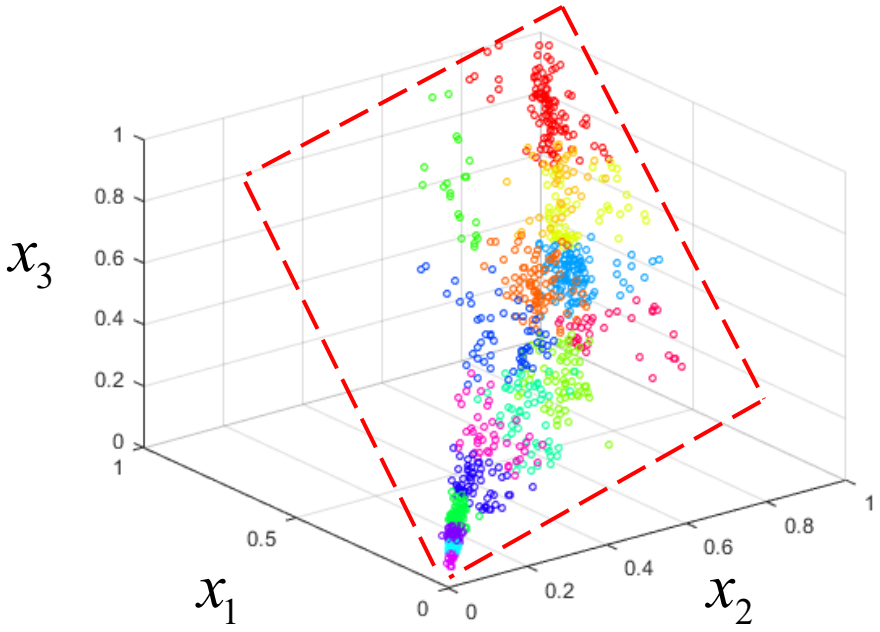
$x^{(2)} \in \mathbb{R}^2 \rightarrow z^{(2)} \in \mathbb{R}$

$\vdots$

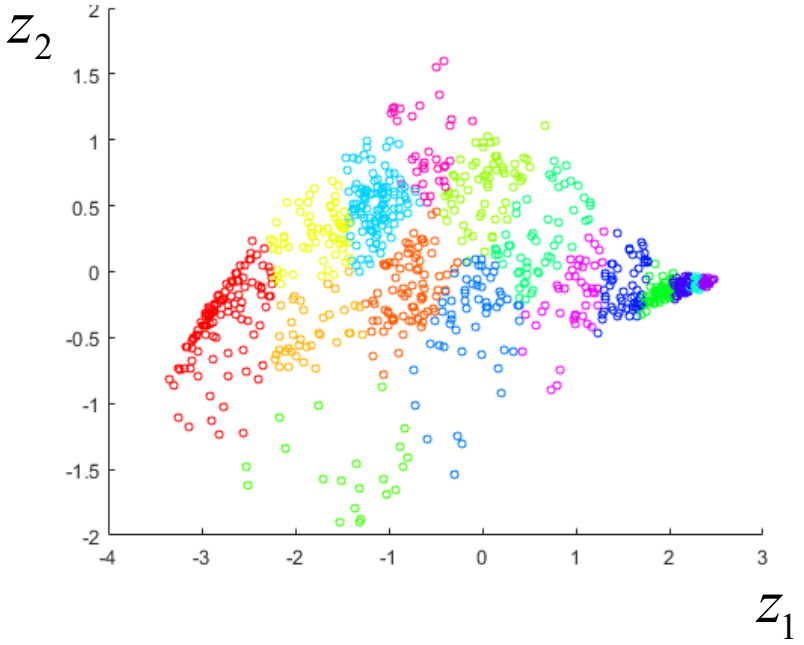
$x^{(m)} \in \mathbb{R}^2 \rightarrow z^{(m)} \in \mathbb{R}$

❖ Motivation. Data Compression and Visualization.

- Reduce data from 3D to 2D



$x^{(i)} \in \mathbb{R}^3$



$z^{(i)} \in \mathbb{R}^2$

Dimensionality reduction

There are two main categories of dimensionality reduction techniques: Feature Selection and Feature Extraction.

Feature selection methods choose a subset of the original features and discard the rest. You are essentially "selecting" the best features to keep.

Feature extraction methods create a new, smaller set of features that are combinations of the original ones. These new features are called "components" or "latent variables."

Dimensionality reduction: Feature Selection

Filter Methods

These methods are applied as a preprocessing step. They score each feature based on a statistical measure and then keep the features with the highest scores. They are fast and independent of the machine learning model you choose later.

Examples:

- **Chi-Squared Test:** Used for categorical features to see how well they predict a categorical target.

- **Correlation Coefficient:** You can calculate the correlation between each feature and the target variable and discard features with low correlation. You can also remove features that are highly correlated with each other (to avoid multicollinearity).

Dimensionality reduction: Feature Selection

Wrapper Methods

These methods use a specific machine learning model to evaluate the usef ulness of a subset of features. They "wrap" the feature selection process ar ound the model, treating it like a black box to score different feature com binations. They are more accurate than filter methods but much slower.

Examples:

- **Forward Selection:** Start with no features. Add features one by one, ke eping the one that improves the model the most at each step, until ad ding new features no longer improves performance.
- **Backward Elimination:** Start with all features. Remove features one by one, deleting the one that impacts performance the least at each step, until removing features starts to significantly hurt performance.

Dimensionality reduction: Feature Selection

Embedded Methods

These methods perform feature selection as part of the model traini ng process itself. They are a good compromise between the accurac y of wrapper methods and the speed of filter methods.

Examples:

- **LASSO (L1) Regularization:** This is a powerful technique used in linear models. It adds a penalty to the model's cost function that is proportional to the absolute value of the feature coefficients. T his has the effect of shrinking the coefficients of less important f eatures down to exactly zero, effectively removing them from the model.
- **Ridge (L2) Regularization:** Similar to LASSO, but it shrinks coeffi cients close to zero without setting them exactly to zero. It's less of a feature selection method and more of a way to reduce mod el complexity.

Dimensionality reduction: Feature Extraction

Principal Component Analysis (PCA)

PCA is the most popular dimensionality reduction technique. It's an unsupervised method that transforms the original features into a ne w set of uncorrelated features called principal components. These c omponents are ordered so that the first few retain most of the varia nce (i.e., information) present in the original dataset.

- **Key Idea:** It finds the directions of maximum variance in the data and projects the data onto a new, lower-dimensional subspace wi thout losing much information.
- **Pros:** Very effective at reducing dimensions and handling multico llinearity.
- **Cons:** The new principal components are linear combinations of t he original features, making them difficult to interpret. You lose t he original meaning of your features.

Dimensionality reduction: Feature Extraction

Linear Discriminant Analysis (LDA)

LDA is a supervised technique, meaning it considers the target variable (class labels). While PCA tries to find the axes with maximum variance, LDA tries to find the axes that maximize the separation between multiple classes.

- **Key Idea:** It projects data onto a lower-dimensional space in a way that makes different classes as distinct as possible.
- **When to Use:** It's an excellent choice for classification problems when you want to reduce dimensions in a way that helps your model distinguish between classes more easily.

4

Principal Component Analysis

Principal Component Analysis

Step 1: Standardize the Data

- PCA is affected by scale. Standardizing ensures each feature has mean = 0 and SD = 1.

Step 2: Calculate the Covariance Matrix

- Captures the relationship between features (e.g., how X varies with Y, etc.)

Step 3: Calculate Eigenvectors and Eigenvalues

- Eigenvectors define the directions (principal components).Eigenvalues indicate the magnitude (importance) of those directions.

Step 4: Select Principal Components

- Choose the top eigenvectors (PC1, PC2,…) for projecting data to lower dimension.

Step 5: Transform the Data onto the New Plane

Multiply the original standardized data by the selected eigenvectors.

A 3D to 2D Numerical Example

PCA Example

Let's imagine we have data for four different products with three features: 'Feature X' (e.g., weight), 'Feature Y' (e.g., durability), and 'Feature Z' (e.g., user rating).

Product	Feature X	Feature Y	Feature Z
A	4	1	1
B	5	3	2
C	8	5	4
D	9	7	5

PCA Example

Step 1: Standardize the Data

First, we scale the data so that each feature has a mean of 0 and a standard deviation of 1. This prevents features with larger scales from dominating the analysis.

1a. Calculate the mean of each feature:

Product	Feature X	Feature Y	Feature Z
A	4	1	1
B	5	3	2
C	8	5	4
D	9	7	5

Mean of X: (4+5+8+9)/4=6.5

Mean of Y: (1+3+5+7)/4=4.0

Mean of Z: (1+2+4+5)/4=3.0

1b. Calculate the standard deviation of each feature:

Std. Dev. of X: ≈2.38

Std. Dev. of Y: ≈2.58

Std. Dev. of Z: ≈1.83

PCA Example

1c. Apply the standardization formula:

(value−mean)/std_dev

This gives us our new standardized dataset:

Product	Feature X	Feature Y	Feature Z
A	4	1	1
B	5	3	2
C	8	5	4
D	9	7	5

Product	Standardized X	Standardized Y	Standardized Z
A	-1.05	-1.16	-1.09
B	-0.63	-0.39	-0.55
C	0.63	0.39	0.55
D	1.05	1.16	1.09

PCA Example

Product	Standardized X	Standardized Y	Standardized Z
A	-1.05	-1.16	-1.09
B	-0.63	-0.39	-0.55
C	0.63	0.39	0.55
D	1.05	1.16	1.09

Step 2: Calculate the Covariance Matrix

Next, we calculate the 3x3 covariance matrix from the standardized data. This matrix shows how the variables relate to each other. For standardized data, the variance of each variable (the diagonal) is 1.

The resulting Covariance Matrix will look something like this:

$$\begin{pmatrix} 1.0 & Cov(X,Y) & Cov(X,Z) \\ Cov(Y,X) & 1.0 & Cov(Y,Z) \\ Cov(Z,X) & Cov(Z,Y) & 1.0 \end{pmatrix} = \begin{pmatrix} 1.0 & 0.99 & 0.99 \\ 0.99 & 1.0 & 1.0 \\ 0.99 & 1.0 & 1.0 \end{pmatrix}$$

Note: The high covariance values indicate our features are strongly correlated, making this a good candidate for PCA.

PCA Example

Product	Standardized X	Standardized Y	Standardized Z
A	-1.05	-1.16	-1.09
B	-0.63	-0.39	-0.55
C	0.63	0.39	0.55
D	1.05	1.16	1.09

Step 3: Calculate Eigenvectors and Eigenvalues

This step involves decomposing the covariance matrix to find the principal components.

Eigenvectors: The directions of the new axes. For a 3D dataset, we get three eigenvectors.

Eigenvalues: The amount of variance captured by each eigenvector.

For our covariance matrix, the linear algebra calculations give us three eigenvalues and their corresponding eigenvectors:

- Eigenvalue 1(λ_1)=2.98 -> Eigenvector 1 (PC1) = (0.58, 0.58, 0.58)
- Eigenvalue 2(λ_2)=0.02 -> Eigenvector 2 (PC2) = (-0.71, 0.10, 0.70)
- Eigenvalue 3(λ_3)=0.00 -> Eigenvector 3 (PC3) = (0.41, -0.81, 0.41)

PCA Example

Product	Standardized X	Standardized Y	Standardized Z
A	-1.05	-1.16	-1.09
B	-0.63	-0.39	-0.55
C	0.63	0.39	0.55
D	1.05	1.16	1.09

Step 4: Select Principal Components

We rank the eigenvectors by their eigenvalues from highest to lowest. The vectors with the highest eigenvalues capture the most information.

PC1 (Eigenvalue = 2.98)

PC2 (Eigenvalue = 0.02)

PC3 (Eigenvalue = 0.00)

Since our goal is to reduce the data from 3D to 2D, we will select the top two principal components: **PC1** and **PC2**.

Let's check how much variance these two components retain:

- Total Variance = 2.98+0.02+0.00=3.0
- Explained by PC1 + PC2 = (2.98+0.02)/3.0=3.0/3.0=100%

In this case, our top two components capture virtually all the information from the original dataset.

PCA Example

Product	Standardized X	Standardized Y	Standardized Z
A	-1.05	-1.16	-1.09
B	-0.63	-0.39	-0.55
C	0.63	0.39	0.55
D	1.05	1.16	1.09

Step 5: Transform the Data onto the New 2D Plane

Finally, we project the standardized 3D data onto our two chosen principal components (PC1 and PC2) to get the new 2D coordinates.

We do this by taking the dot product of each standardized data point with each of our chosen eigenvectors.

For Product A:

- New X-coordinate (on PC1):

$(-1.05*0.58)+(-1.16*0.58)+(-1.09*0.58)=-0.61-0.67-0.63=\mathbf{-1.91}$

- New Y-coordinate (on PC2):

$(-1.05*-0.71)+(-1.16*0.10)+(-1.09*0.70)=0.75-0.12-0.76=\mathbf{-0.13}$

Repeating this for all data points gives us our final 2D dataset.

PCA Example

Result

The original 3D data has been successfully transformed into a 2D representation.

Product	Original (X, Y, Z)	New 2D Coordinates (PC1, PC2)
A	(4, 1, 1)	(-1.91, -0.13)
B	(5, 3, 2)	(-0.91, 0.04)
C	(8, 5, 4)	(0.91, -0.04)
D	(9, 7, 5)	(1.91, 0.13)

These new coordinates can now be easily plotted on a 2D scatter plot to visualize the relationships between the prod ucts.

Application: Image compression

Original Image

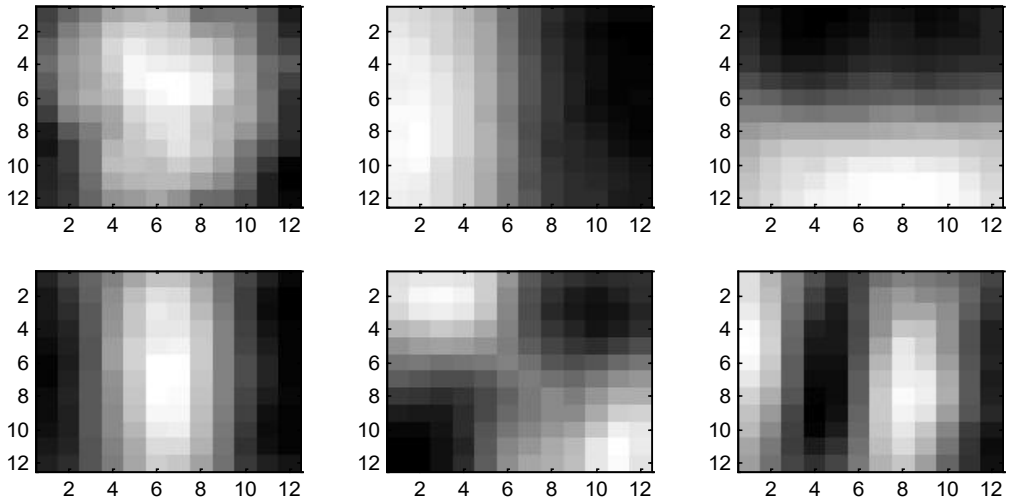
- Divide the original 372x492 image into patches:
  - Each patch is an instance that contains 12x12 pixels on a grid
- View each as a 144-D vector

PCA compression: 144D → 60D

PCA compression: 144D → 16D

PCA compression: 144D) 6D

6 most important eigenvectors



PCA compression: 144D → 3D

PCA compression: 144D → 1D